

Cloud Spanner：使用 C# 建立遊戲排行榜

來源：<https://codelabs.developers.google.com/codelabs/cloud-spanner-leaderboard?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何使用含有修訂時間戳記欄的 Cloud Spanner 資料庫資料表，建立遊戲排行榜。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 設定 Cloud Spanner 執行個體](#)
4. [4. 建立資料庫和結構定義](#)
5. [5. 載入資料](#)
6. [6. 執行排行榜查詢](#)
7. [7. 清除所用資源](#)
8. [8. 恭喜！](#)

1. 總覽

[Google Cloud Spanner](#) 是全代管的關聯式資料庫服務，可水平擴充、遍及全球，提供 ACID 交易和 SQL 語意，同時兼顧效能和高可用性。

在本實驗室中，您將學習如何設定 Cloud Spanner 執行個體。您將逐步建立可用於遊戲排行榜的資料庫和結構定義。首先，您要建立「Players」資料表來儲存球員資訊，以及「Scores」資料表來儲存球員得分。

接著，您會將範例資料填入資料表。接著，您將執行一些前十大範例查詢，最後刪除執行個體來釋出資源，完成實驗室。

課程內容

- 如何設定 Cloud Spanner 執行個體。
- 如何建立資料庫和資料表。
- 如何使用修訂時間戳記欄。
- 如何將資料載入 Cloud Spanner 資料庫資料表，並加上時間戳記。
- 如何查詢 Cloud Spanner 資料庫。
- 如何刪除 Cloud Spanner 執行個體。

軟硬體需求

- [Chrome](#) 或 [Firefox](#) 瀏覽器。
-

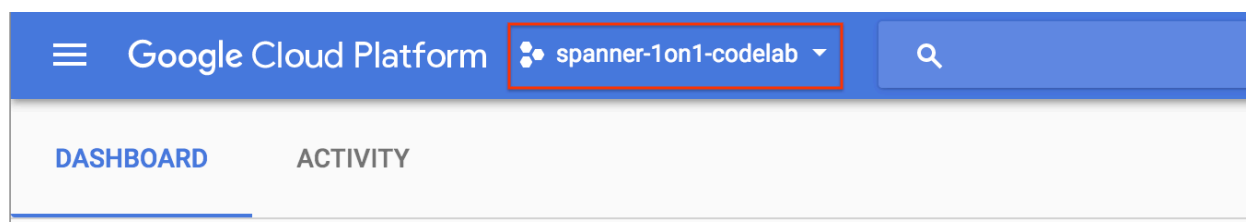
步驟 2 · 約 5 分鐘

2. 設定和需求

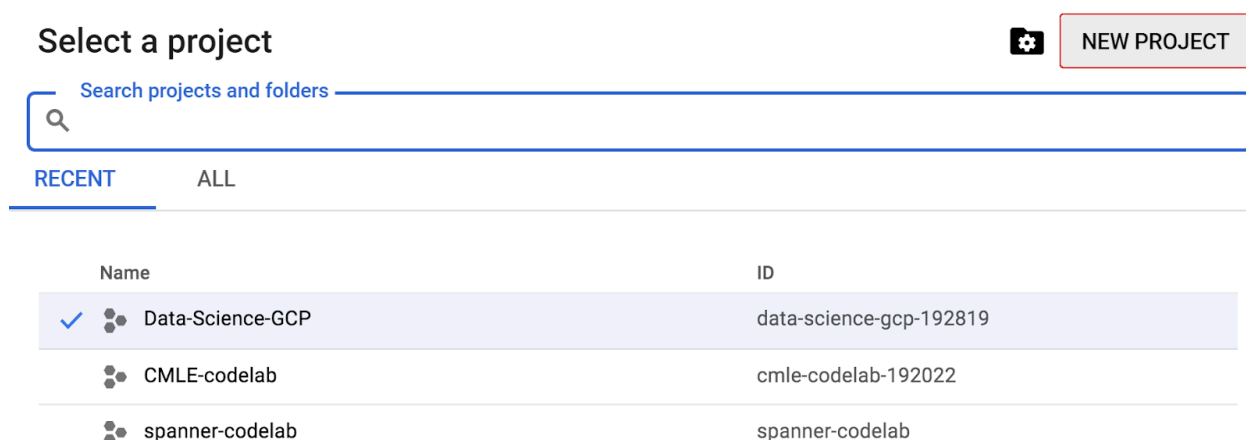
自修實驗室環境設定

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案。

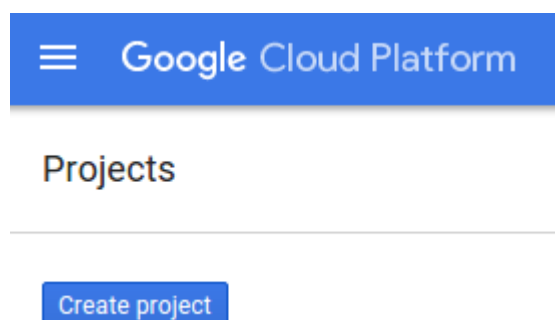
如果您已有專案，請按一下主控台左上方的專案選取下拉式選單：



然後在隨即顯示的對話方塊中，按一下「NEW PROJECT」(新專案) 按鈕，建立新專案：



如果您還沒有專案，應該會看到如下對話方塊，請建立第一個專案：



在隨後的專案建立對話方塊中，您可以輸入新專案的詳細資料：

New Project

Project name ?

spanner-1on1-codelab

Your project ID will be spanner-1on1-codelab ? [Edit](#)

Billing account ?

Choose account

Create

Cancel

請記住專案 ID，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

接下來，如果尚未啟用，請在開發人員控制台中[啟用帳單](#)，以便使用 Google Cloud 資源，並[啟用 Cloud Spanner API](#)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#)

or

[Manage billing accounts](#)

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。Google Cloud Spanner 的定價說明文件請參閱[這裡](#)。

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**，因此本程式碼研究室應完全免費。

Google Cloud Shell 設定

雖然可以透過筆電遠端操作 Google Cloud 和 Spanner，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是在雲端執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



gcl

Bcz

TXI

即可 (佈建並連線至環境的作業需要一些時間才能完成)。



JjEuRXGg0AYYIY6QZ8d-66gx_Mtc-_jDE9ijmbXLJSAXFvJt-qUpNtsBsYjNpv2W6BQSRdc1D-ARINNQ-1EkWUhZ-iUK-FUCZhJ-NtjvIEx9pIkE-246DomWuCFiGHK78DgoeWkHRw



Screen Shot 2017-06-14 at 10.13.43 PM.png

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



```
158fNPwSxsFqz9YbtJVZes8viTS3d1bV4CVhij3XPxuzVFOtTObnwsphlm6lYGmg
FaLrZU7XHAg_ZYoCrgombMRR3h-
eolLPcvO351c5iBv506B3ZwghZoiRg6cz23Qw
```

Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

摘要

在這個步驟中，您會設定環境。

下一步

接下來，您將設定 Cloud Spanner 執行個體。

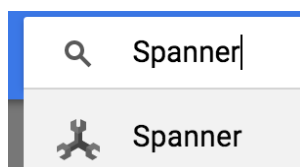
步驟 3 · 約 3 分鐘

3. 設定 Cloud Spanner 執行個體

在本步驟中，我們將為這個程式碼研究室設定 Cloud Spanner 執行個體。在左上方的「漢堡」選單中搜尋 Spanner 項目



，或按下「/」並輸入「Spanner」來搜尋



接著，按一下

Create instance

並填寫表單，輸入執行個體的執行個體名稱 **cloudspanner-leaderboard**，選擇設定 (選取區域執行個體)，並設定節點數量。在本程式碼研究室中，我們只需要 1 個節點。如要使用實際工作環境執行個體，並符合 Cloud Spanner 服務等級協議的資格，您需要在 Cloud Spanner 執行個體中執行 3 個以上的節點。

最後，按一下「建立」，幾秒內您就能使用 Cloud Spanner 執行個體。

Create an instance

Instance name

For display purposes only.

Instance ID

Unique identifier for instance. Permanent.

Configuration

Determines where your data and nodes are located. Affects cost, performance, and replication. This choice is permanent. Select a configuration to view its details.



Regional



Multi-region

Nodes

Add nodes to increase data throughput and queries per second (QPS). Affects billing.

 [Node guidance](#)

Cost

Storage cost depends on GB stored per month. Nodes cost is an hourly charge for the number of nodes in your instance. [Learn more](#)

Nodes cost

\$0.90 per hour

Storage cost

\$0.30 per GB/month

在下一個步驟中，我們將使用 C# 用戶端程式庫，在新執行個體中建立資料庫和結構定義。

步驟 4 · 約 10 分鐘

4. 建立資料庫和結構定義

在這個步驟中，我們要建立資料庫範例和結構定義。

我們將使用 C# 用戶端程式庫建立兩個資料表：一個是 Players 資料表，用於儲存玩家資訊；另一個是 Scores 資料表，用於儲存玩家分數。為此，我們將逐步說明如何在 Cloud Shell 中建立 C# 控制台應用程式。

首先，請在 Cloud Shell 中輸入下列指令，從 GitHub 複製本程式碼研究室的範例程式碼：

```
git clone https://github.com/GoogleCloudPlatform/dotnet-docs-samples.git
```

然後將目錄變更為「applications」目錄，您將在其中建立應用程式。

```
cd dotnet-docs-samples/applications/
```

本程式碼研究室所需的所有程式碼都位於現有的 `dotnet-docs-samples/applications/leaderboard` 目錄中，做為名為 `Leaderboard` 的可執行 C# 應用程式，供您在完成本程式碼研究室時做為參考。我們會建立新目錄，並分階段建構排行榜應用程式的副本。

建立名為「codelab」的應用程式新目錄，然後使用下列指令將目錄變更為該目錄：

```
mkdir codelab && cd $_
```

使用下列指令建立名為「Leaderboard」的新 .NET C# 控制台應用程式：

```
dotnet new console -n Leaderboard
```

這個指令會建立簡單的控制台應用程式，其中包含兩個主要檔案：專案檔案 `Leaderboard.csproj` 和程式檔案 `Program.cs`。

現在就來執行。將目錄變更為新建立的 `Leaderboard` 目錄，也就是應用程式所在的目錄：

```
cd Leaderboard
```

然後輸入下列指令來執行。

```
dotnet run
```

您應該會看到應用程式輸出內容「Hello World!」。

現在請編輯 `Program.cs`，更新控制台應用程式，使用 C# Spanner 用戶端程式庫建立排行榜，其中包含 Players 和 Scores 兩個資料表。您可以在 Cloud Shell 編輯器中執行這項操作：

按一下下方標示的圖示，開啟 Cloud Shell 編輯器：

The screenshot shows the Google Cloud Platform console interface. At the top, the header bar includes the Google Cloud Platform logo, the text "spanner-codelab", and various utility icons. Below the header, the "Spanner" section is active, displaying a table of instances under the "Instances" tab. The table has columns for Name, Configuration, Nodes, and Labels. One instance, "cloudspanner-leaderboard", is listed with configuration "us-west1" and 1 node. A "CREATE INSTANCE" button is visible. Below the table, a Cloud Shell terminal window is open, showing a welcome message and a prompt. The terminal window's toolbar includes a red box highlighting the edit icon.

Name	Configuration	Nodes	Labels
cloudspanner-leaderboard	us-west1	1	

```
Welcome to Cloud Shell! Type "help" to get started.
ononon@ononon:~$
```

接著，在 Cloud Shell 編輯器中開啟 `Program.cs` 檔案，然後將下列 C# 應用程式碼貼入 `Program.cs` 檔案，取代檔案中的現有程式碼，建立 `leaderboard` 資料庫和 `Players` 與 `Scores` 資料表：

```

using System;
using System.Threading.Tasks;
using Google.Cloud.Spanner.Data;
using CommandLine;

namespace GoogleCloudSamples.Leaderboard
{
    [Verb("create", HelpText = "Create a sample Cloud Spanner database "
        + "along with sample 'Players' and 'Scores' tables in your project.")]
    class CreateOptions
    {
        [Value(0, HelpText = "The project ID of the project to use "
            + "when creating Cloud Spanner resources.", Required = true)]
        public string projectId { get; set; }
        [Value(1, HelpText = "The ID of the instance where the sample database "
            + "will be created.", Required = true)]
        public string instanceId { get; set; }
        [Value(2, HelpText = "The ID of the sample database to create.",
            Required = true)]
        public string databaseId { get; set; }
    }

    public class Program
    {
        enum ExitCode : int
        {
            Success = 0,
            InvalidParameter = 1,
        }

        public static object Create(string projectId,
            string instanceId, string databaseId)
        {
            var response =
                CreateAsync(projectId, instanceId, databaseId);
            Console.WriteLine("Waiting for operation to complete...");
            response.Wait();
            Console.WriteLine($"Operation status: {response.Status}");
            Console.WriteLine($"Created sample database {databaseId} on "
                + $"instance {instanceId}");
            return ExitCode.Success;
        }

        public static async Task CreateAsync(
            string projectId, string instanceId, string databaseId)
        {
            // Initialize request connection string for database creation.
            string connectionString =
                $"Data Source=projects/{projectId}/instances/{instanceId}";
            using (var connection = new SpannerConnection(connectionString))
            {
                string createStatement = $"CREATE DATABASE `{databaseId}`";
            }
        }
    }
}

```

```

        string[] createTableStatements = new string[] {
            // Define create table statement for Players table.
            @"CREATE TABLE Players(
                PlayerId INT64 NOT NULL,
                PlayerName STRING(2048) NOT NULL
            ) PRIMARY KEY(PlayerId)",
            // Define create table statement for Scores table.
            @"CREATE TABLE Scores(
                PlayerId INT64 NOT NULL,
                Score INT64 NOT NULL,
                Timestamp TIMESTAMP NOT NULL OPTIONS(allow_commit_timestamp=true)
            ) PRIMARY KEY(PlayerId, Timestamp),
                INTERLEAVE IN PARENT Players ON DELETE NO ACTION" };
        // Make the request.
        var cmd = connection.CreateDdlCommand(
            createStatement, createTableStatements);
        try
        {
            await cmd.ExecuteNonQueryAsync();
        }
        catch (SpannerException e) when
            (e.ErrorCode == ErrorCode.AlreadyExists)
        {
            // OK.
        }
    }
}

public static int Main(string[] args)
{
    var verbMap = new VerbMap<object>();
    verbMap
        .Add((CreateOptions opts) => Create(
            opts.projectId, opts.instanceId, opts.databaseId))
        .NotParsedFunc = (err) => 1;
    return (int)verbMap.Run(args);
}
}
}

```

為清楚呈現程式碼，以下是程式的圖表，並標示主要元件：

Import Namespace
Dependencies

Define Program
Namespace

Verbmap
Defining
"create"
command
with three
arguments

Program's Class
Declaration

Define Exit Code
Enumerator

"Create" Method
to call the
"CreateAsync"
Method

"CreateAsync"
Method that
creates the
Database
and two
Tables
asynchronously

```
using System;
using System.Threading.Tasks;
using Google.Cloud.Spanner.Data;
using CommandLine;

namespace GoogleCloudSamples.Leaderboard
{
    [Verb("create", HelpText = "Create a sample Cloud Spanner database "
        + "along with sample 'Players' and 'Scores' tables in your project.")]
    class CreateOptions
    {
        [Value(0, HelpText = "The project ID of the project to use "
            + "when creating Cloud Spanner resources.", Required = true)]
        public string projectId { get; set; }
        [Value(1, HelpText = "The ID of the instance where the sample database "
            + "will be created.", Required = true)]
        public string instanceId { get; set; }
        [Value(2, HelpText = "The ID of the sample database to create.",
            Required = true)]
        public string databaseId { get; set; }
    }

    public class Program
    {
        enum ExitCode : int
        {
            Success = 0,
            InvalidParameter = 1,
        }

        public static object Create(string projectId,
            string instanceId, string databaseId)
        {
            var response =
                CreateAsync(projectId, instanceId, databaseId);
            Console.WriteLine("Waiting for operation to complete...");
            response.Wait();
            Console.WriteLine($"Operation status: {response.Status}");
            Console.WriteLine($"Created sample database {databaseId} on "
                + $"instance {instanceId}");
            return ExitCode.Success;
        }

        public static async Task CreateAsync(
            string projectId, string instanceId, string databaseId)
        {
            // Initialize request connection string for database creation.
            string connectionString =
                $"Data Source=projects/{projectId}/instances/{instanceId}";
            using (var connection = new SpannerConnection(connectionString))
            {
                string createStatement = $"CREATE DATABASE `{databaseId}`";
                string[] createTableStatements = new string[] {
                    // Define create table statement for Players table.
                    @"CREATE TABLE Players(
                        PlayerId INT64 NOT NULL,
                        PlayerName STRING(2048) NOT NULL
                    ) PRIMARY KEY(PlayerId)",
                    // Define create table statement for Scores table.
                    @"CREATE TABLE Scores(
                        PlayerId INT64 NOT NULL,
                        Score INT64 NOT NULL,
                        Timestamp TIMESTAMP NOT NULL OPTIONS(allow_commit_timestamp=true)
                    ) PRIMARY KEY(PlayerId, Timestamp),
                    INTERLEAVE IN PARENT Players ON DELETE NO ACTION" };
                // Make the request.
                var cmd = connection.CreateDdlCommand(
                    createStatement, createTableStatements);
                try
                {
                    await cmd.ExecuteNonQueryAsync();
                }
                catch (SpannerException e) when
                    (e.ErrorCode == ErrorCode.AlreadyExists)
                {
                    // OK.
                }
            }
        }
    }

    public static int Main(string[] args)
    {
    }
}
```

Main Method
serves as the
Program
entry point

```
public static int Main(string[] args)
{
    var verbMap = new VerbMap<object>();
    verbMap
        .Add((CreateOptions opts) => Create(
            opts.projectId, opts.instanceId, opts.databaseId))
        .NotParsedFunc = (err) => 1;
    return (int)verbMap.Run(args);
}
```

您可以使用 `dotnet-docs-samples/applications/leaderboard/step4` 目錄中的 `Program.cs` 檔案，查看啟用 `create` 指令後 `Program.cs` 檔案的範例。

接著，使用 Cloud Shell 編輯器開啟並編輯程式的專案檔案 `Leaderboard.csproj`，將其更新為下列程式碼。請務必使用 Cloud Shell 編輯器的「File」(檔案) 選單儲存所有變更。

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>netcoreapp3.1</TargetFramework>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Google.Cloud.Spanner.Data" Version="3.3.0" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\..\..\commandlineutil\Lib\CommandLineUtil.csproj" />
  </ItemGroup>

</Project>
```

這項變更新增了 C# Spanner Nuget 套件 `Google.Cloud.Spanner.Data` 的參照，我們需要與 Cloud Spanner API 互動。這項變更也會新增對 `CommandLineUtil` 專案的參照，該專案是 `dotnet-doc-samples` Github 存放區的一部分，並為開放原始碼 `CommandLineParser` 提供實用的「verbmap」擴充功能，方便處理主控台應用程式的指令列輸入內容。

您可以使用 `dotnet-docs-samples/applications/leaderboard/step4` 目錄中的 `Leaderboard.csproj` 檔案，查看啟用 `create` 指令後 `Leaderboard.csproj` 檔案的範例。

現在可以執行更新後的範例了。輸入下列內容，查看更新後應用程式的預設回應：

```
dotnet run
```

您會看到如下所示的輸出：

```
Leaderboard 1.0.0
Copyright (C) 2018 Leaderboard
```

```
ERROR(S):
```

```
No verb selected.
```

```
create      Create a sample Cloud Spanner database along with sample 'Players' and
'Scores' tables in your project.
```

```
help        Display more information on a specific command.
```

```
version     Display version information.
```

從這個回應中，我們可以得知這是 `Leaderboard` 應用程式，可透過下列三種指令之一執行：`create`、`help` 和 `version`。

我們來試試 `create` 指令，建立 Spanner 資料庫和資料表。執行不含引數的指令，即可查看指令的預期引數。

```
dotnet run create
```

畫面會顯示類似以下的回應：

```
Leaderboard 1.0.0
Copyright (C) 2018 Leaderboard
```

```
ERROR(S):
```

```
A required value not bound to option name is missing.
```

```
--help      Display this help screen.
```

```
--version   Display version information.
```

```
value pos. 0 Required. The project ID of the project to use when creating Cloud
Spanner resources.
```

```
value pos. 1 Required. The ID of the instance where the sample database will be
created.
```

```
value pos. 2 Required. The ID of the sample database to create.
```

從這裡可以看到 `create` 指令的預期引數為專案 ID、執行個體 ID 和資料庫 ID。

現在請執行下列指令。**請務必**將 `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run create PROJECT_ID cloudspanner-leaderboard leaderboard
```

幾秒後，您應該會看到類似以下的回應：

Waiting for operation to complete...

Operation status: RanToCompletion

Created sample database leaderboard on instance cloudspanner-leaderboard

在 Cloud 控制台的「Cloud Spanner」部分，左側選單應該會顯示新的資料庫和資料表。

← Instance

cloudspanner-leaderboard

leaderboard

Query

Players

Scores

← Table details

QUERY

CREATE INDEX

Players

Schema

IndexesData

Column	Type	Nullable
 PlayerId	INT64	No
PlayerName	STRING(2048)	No

Show equivalent DDL

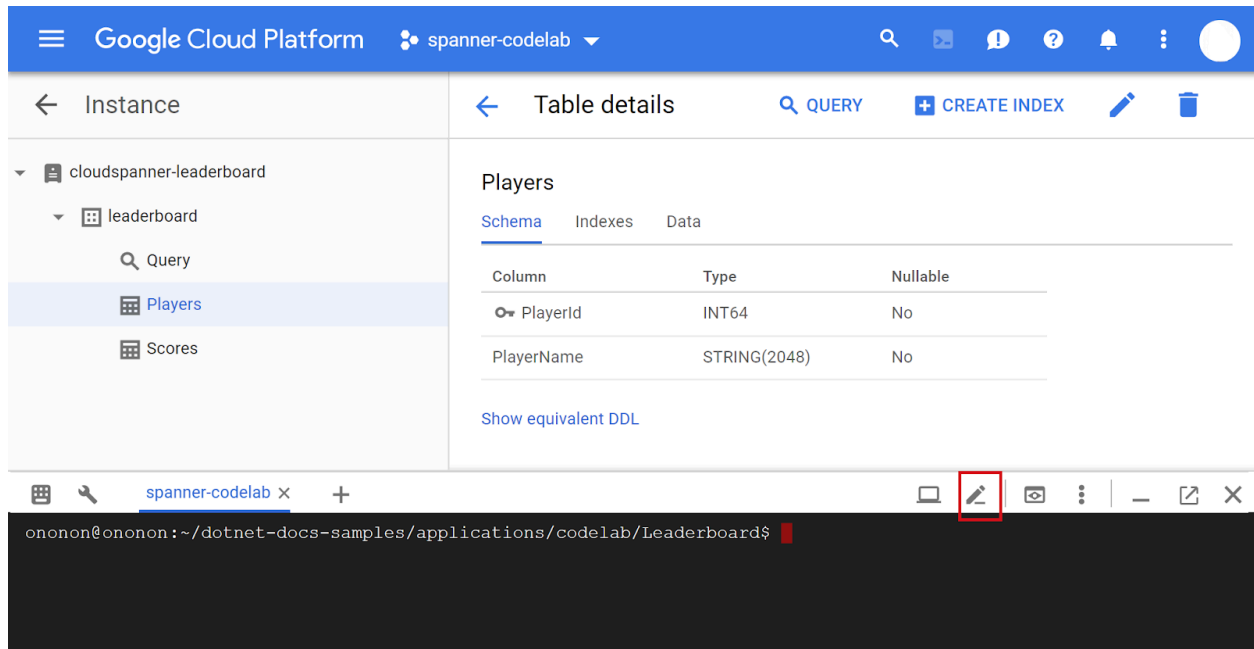
在下一個步驟中，我們會更新應用程式，將一些資料載入新的資料庫。

步驟 5 · 約 12 分鐘

5. 載入資料

現在我們有一個名為 `leaderboard` 的資料庫，其中包含兩個資料表：`Players` 和 `Scores`。現在讓我們使用 C# 用戶端程式庫，在 `Players` 表格中填入玩家，並在 `Scores` 表格中填入每位玩家的隨機分數。

按一下下方標示的圖示，開啟 Cloud Shell 編輯器：



接著，在 Cloud Shell 編輯器中編輯 `Program.cs` 檔案，加入 `insert` 指令，將 100 名玩家插入 `Players` 資料表，或為 `Players` 資料表中的每位玩家，在 `Scores` 資料表中插入 4 個隨機分數。

首先，請在現有 `create` 指令區塊下方的「Verbmap」中，新增 `insert` 指令區塊：

```
[Verb("insert", HelpText = "Insert sample 'players' records or 'scores' records "
    + "into the database.")]
class InsertOptions
{
    [Value(0, HelpText = "The project ID of the project to use "
        + "when managing Cloud Spanner resources.", Required = true)]
    public string projectId { get; set; }
    [Value(1, HelpText = "The ID of the instance where the sample database resides.",
        Required = true)]
    public string instanceId { get; set; }
    [Value(2, HelpText = "The ID of the database where the sample database resides.",
        Required = true)]
    public string databaseId { get; set; }
    [Value(3, HelpText = "The type of insert to perform, 'players' or 'scores'.",
        Required = true)]
    public string insertType { get; set; }
}
```

接著，在現有的 `CreateAsync` 方法下方新增下列 `Insert`、`InsertPlayersAsync` 和 `InsertScoresAsync` 方法：

```

public static object Insert(string projectId,
    string instanceId, string databaseId, string insertType)
{
    if (insertType.ToLower() == "players")
    {
        var responseTask =
            InsertPlayersAsync(projectId, instanceId, databaseId);
        Console.WriteLine("Waiting for insert players operation to complete...");
        responseTask.Wait();
        Console.WriteLine($"Operation status: {responseTask.Status}");
    }
    else if (insertType.ToLower() == "scores")
    {
        var responseTask =
            InsertScoresAsync(projectId, instanceId, databaseId);
        Console.WriteLine("Waiting for insert scores operation to complete...");
        responseTask.Wait();
        Console.WriteLine($"Operation status: {responseTask.Status}");
    }
    else
    {
        Console.WriteLine("Invalid value for 'type of insert'. "
            + "Specify 'players' or 'scores'.");
        return ExitCode.InvalidParameter;
    }
    Console.WriteLine($"Inserted {insertType} into sample database "
        + $"{databaseId} on instance {instanceId}");
    return ExitCode.Success;
}

public static async Task InsertPlayersAsync(string projectId,
    string instanceId, string databaseId)
{
    string connectionString =
        $"Data Source=projects/{projectId}/instances/{instanceId}"
        + $"/databases/{databaseId}";

    long numberOfPlayers = 0;
    using (var connection = new SpannerConnection(connectionString))
    {
        await connection.OpenAsync();
        await connection.RunWithRetriableTransactionAsync(async (transaction) =>
        {
            // Execute a SQL statement to get current number of records
            // in the Players table to use as an incrementing value
            // for each PlayerName to be inserted.
            var cmd = connection.CreateSelectCommand(
                @"SELECT Count(PlayerId) as PlayerCount FROM Players");
            numberOfPlayers = await cmd.ExecuteScalarAsync<long>();
            // Insert 100 player records into the Players table.
            SpannerBatchCommand cmdBatch = connection.CreateBatchDmlCommand();
            for (int i = 0; i < 100; i++)

```

```

        {
            numberOfPlayers++;
            SpannerCommand cmdInsert = connection.CreateDmlCommand(
                "INSERT INTO Players "
                + "(PlayerId, PlayerName) "
                + "VALUES (@PlayerId, @PlayerName)",
                new SpannerParameterCollection {
                    {"PlayerId", SpannerDbType.Int64},
                    {"PlayerName", SpannerDbType.String}});
            cmdInsert.Parameters["PlayerId"].Value =
                Math.Abs(Guid.NewGuid().GetHashCode());
            cmdInsert.Parameters["PlayerName"].Value =
                $"Player {numberOfPlayers}";
            cmdBatch.Add(cmdInsert);
        }
        await cmdBatch.ExecuteNonQueryAsync();
    });
}

Console.WriteLine("Done inserting player records...");
}

public static async Task InsertScoresAsync(
    string projectId, string instanceId, string databaseId)
{
    string connectionString =
        $"Data Source=projects/{projectId}/instances/{instanceId}"
        + $" /databases/{databaseId}";

    // Insert 4 score records into the Scores table for each player
    // in the Players table.
    using (var connection = new SpannerConnection(connectionString))
    {
        await connection.OpenAsync();
        await connection.RunWithRetriableTransactionAsync(async (transaction) =>
        {
            Random r = new Random();
            bool playerRecordsFound = false;
            SpannerBatchCommand cmdBatch =
                connection.CreateBatchDmlCommand();
            var cmdLookup =
                connection.CreateSelectCommand("SELECT * FROM Players");
            using (var reader = await cmdLookup.ExecuteReaderAsync())
            {
                while (await reader.ReadAsync())
                {
                    playerRecordsFound = true;
                    for (int i = 0; i < 4; i++)
                    {
                        DateTime randomTimestamp = DateTime.Now
                            .AddYears(r.Next(-2, 1))
                            .AddMonths(r.Next(-12, 1))
                            .AddDays(r.Next(-28, 0))

```

```

        .AddHours(r.Next(-24, 0))
        .AddSeconds(r.Next(-60, 0))
        .AddMilliseconds(r.Next(-100000, 0));
    SpannerCommand cmdInsert =
    connection.CreateCommand(
        "INSERT INTO Scores "
        + "(PlayerId, Score, Timestamp) "
        + "VALUES (@PlayerId, @Score, @Timestamp)",
        new SpannerParameterCollection {
            {"PlayerId", SpannerDbType.Int64},
            {"Score", SpannerDbType.Int64},
            {"Timestamp",
                SpannerDbType.Timestamp}});
    cmdInsert.Parameters["PlayerId"].Value =
        reader.GetFieldValue<int>("PlayerId");
    cmdInsert.Parameters["Score"].Value =
        r.Next(1000, 1000001);
    cmdInsert.Parameters["Timestamp"].Value =
        randomTimestamp.ToString("o");
    cmdBatch.Add(cmdInsert);
    }
}
if (!playerRecordsFound)
{
    Console.WriteLine("Parameter 'scores' is invalid "
        + "since no player records currently exist. First "
        + "insert players then insert scores.");
    Environment.Exit((int)ExitCode.InvalidParameter);
}
else
{
    await cmdBatch.ExecuteNonQueryAsync();
    Console.WriteLine(
        "Done inserting score records..."
    );
}
}
});
}
}
}

```

接著，如要讓 `insert` 指令正常運作，請將下列程式碼新增至程式的「Main」方法：

```

.Add((InsertOptions opts) => Insert(
    opts.projectId, opts.instanceId, opts.databaseId, opts.insertType))

```

您可以使用 `dotnet-docs-samples/applications/leaderboard/step5` 目錄中的 `Program.cs` 檔案，查看啟用 `insert` 指令後 `Program.cs` 檔案的範例。

現在執行程式，確認新 `insert` 指令已納入程式的可能指令清單。執行下列指令：

```
dotnet run
```

您現在應該會在程式的預設輸出內容中看到 `insert` 指令：

```
Leaderboard 1.0.0
Copyright (C) 2018 Leaderboard

ERROR(S):
  No verb selected.

  create      Create a sample Cloud Spanner database along with sample 'Players' and
'Scores' tables in your project.

  insert      Insert sample 'players' records or 'scores' records into the database.

  help        Display more information on a specific command.

  version     Display version information.
```

現在執行 `insert` 指令，查看輸入引數。輸入下列指令。

```
dotnet run insert
```

這會傳回下列回應：

```
Leaderboard 1.0.0
Copyright (C) 2018 Leaderboard

ERROR(S):
  A required value not bound to option name is missing.

  --help      Display this help screen.

  --version   Display version information.

  value pos. 0 Required. The project ID of the project to use when managing Cloud
Spanner resources.

  value pos. 1 Required. The ID of the instance where the sample database resides.

  value pos. 2 Required. The ID of the database where the sample database resides.

  value pos. 3 Required. The type of insert to perform, 'players' or 'scores'.
```

從回應中可以看出，除了專案 ID、執行個體 ID 和資料庫 ID 之外，還有另一個預期的引數 `value pos. 3`，也就是要執行的「插入類型」。這個引數的值可以是「players」或「scores」。

現在，請使用呼叫 `create` 指令時使用的相同引數值，執行 `insert` 指令，並將「players」新增為額外的「插入類型」引數。**請務必**將 `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run insert PROJECT_ID cloudspanner-leaderboard leaderboard players
```

幾秒後，您應該會看到類似以下的回應：

```
Waiting for insert players operation to complete...
Done inserting player records...
Operation status: RanToCompletion
Inserted players into sample database leaderboard on instance cloudspanner-
leaderboard
```

現在讓我們使用 C# 用戶端程式庫，在 `Scores` 資料表中填入四個隨機分數，以及 `Players` 資料表中每位玩家的時間戳記。

先前執行 `create` 指令時，我們透過下列 SQL 陳述式將 `Scores` 資料表的 `Timestamp` 資料欄定義為「提交時間戳記」資料欄：

```
CREATE TABLE Scores(
  PlayerId INT64 NOT NULL,
  Score INT64 NOT NULL,
  Timestamp TIMESTAMP NOT NULL OPTIONS(allow_commit_timestamp=true)
) PRIMARY KEY(PlayerId, Timestamp),
  INTERLEAVE IN PARENT Players ON DELETE NO ACTION
```

請注意 `OPTIONS(allow_commit_timestamp=true)` 屬性。這會將 `Timestamp` 設為「修訂時間戳記」資料欄，並在特定資料表列上執行 `INSERT` 和 `UPDATE` 作業時，自動填入確切的交易時間戳記。

您也可以將自己的時間戳記值插入「修訂時間戳記」欄，只要插入的值是過去的時間戳記即可。在本程式碼研究室中，我們將這麼做。

現在，請使用呼叫 `create` 指令時使用的相同引數值，執行 `insert` 指令，並將「scores」新增為額外的「插入類型」引數。**請務必**將 `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run insert PROJECT_ID cloudspanner-leaderboard leaderboard scores
```

幾秒後，您應該會看到類似以下的回應：

```
Waiting for insert players operation to complete...
Done inserting player records...
Operation status: RanToCompletion
Inserted players into sample database leaderboard on instance cloudspanner-
leaderboard
```

執行 `insert` 時，如果「插入類型」指定為 `scores`，系統會呼叫 `InsertScoresAsync` 方法，並使用下列程式碼片段插入隨機產生的時間戳記，以及過去發生的日期時間：

```
DateTime randomTimestamp = DateTime.Now
    .AddYears(r.Next(-2, 1))
    .AddMonths(r.Next(-12, 1))
    .AddDays(r.Next(-28, 0))
    .AddHours(r.Next(-24, 0))
    .AddSeconds(r.Next(-60, 0))
    .AddMilliseconds(r.Next(-100000, 0));
...
cmdInsert.Parameters["Timestamp"].Value = randomTimestamp.ToString("o");
```

如要自動在 `Timestamp` 欄位中填入「插入」交易的確切時間戳記，可以改為插入 C# 常數 `SpannerParameter.CommitTimestamp`，如下列程式碼片段所示：

```
cmd.Parameters["Timestamp"].Value = SpannerParameter.CommitTimestamp;
```

資料載入完成後，請驗證我們剛才寫入新資料表的值。首先選取「`leaderboard`」資料庫，然後選取「`Players`」資料表。按一下「`Data`」分頁標籤。您應該會看到資料表 `PlayerId` 和 `PlayerName` 欄中的資料。

← Instance

cloudspanner-leaderboard

leaderboard

Query

Players

Scores

← Table details

Schema

Indexes

Data

Insert

Edit

Delete

Filter by primary key

?

☐ PlayerId 🔑	PlayerName
☐ 378825	Player 46
☐ 18702797	Player 66
☐ 27098435	Player 5
☐ 48936749	Player 74
☐ 56790840	Player 49
☐ 61891198	Player 19
☐ 65206946	Player 34
☐ 70762849	Player 77
☐ 103961126	Player 70

接著，按一下「`Scores`」資料表並選取「`Data`」分頁標籤，確認「`Scores`」資料表也有資料。您應該會看到資料表中的 `PlayerId`、`Timestamp` 和 `Score` 欄位有資料。

← Instance

cloudspanner-leaderboard

leaderboard

Query

Players

Scores

← Table details

🔍

+

✎

🗑

Scores

Schema

Indexes

Data

Insert

Edit

Delete

☰

Filter by primary key

?

☐ PlayerId 🔑	☐ Timestamp 🔑	Score
☐ 378825	2015-08-02T00:00:33.2391574Z	828239
☐ 378825	2015-10-01T23:58:53.4745208Z	455051
☐ 378825	2016-04-22T23:59:00.9733714Z	554265
☐ 378825	2016-06-26T23:59:18.8755464Z	279516
☐ 18702797	2016-09-03T00:00:17.834217Z	79710
☐ 18702797	2017-06-25T23:58:43.9951152Z	862528

非常好！讓我們更新程式，執行一些可用於建立遊戲排行榜的查詢。

6. 執行排行榜查詢

我們已設定資料庫並將資訊載入資料表，現在要使用這些資料建立排行榜。為此，我們需要回答下列四個問題：

1. 哪幾位球員是史上「前十名」？
2. 哪些球員是年度「十大」球員？
3. 哪些玩家是當月的「前十名」？
4. 本週「前十名」的播放器是哪些？

現在來更新程式，執行 SQL 查詢來回答這些問題。

我們會新增 `query` 指令，方便您執行查詢來回答問題，並產生排行榜所需的資訊。

在 Cloud Shell 編輯器中編輯 `Program.cs` 檔案，更新程式以新增 `query` 指令。

首先，請在現有 `insert` 指令區塊下方的「Verbmap」中，新增 `query` 指令區塊：

```
[Verb("query", HelpText = "Query players with 'Top Ten' scores within a specific timespan "
    + "from sample Cloud Spanner database table.")]
class QueryOptions
{
    [Value(0, HelpText = "The project ID of the project to use "
        + "when managing Cloud Spanner resources.", Required = true)]
    public string projectId { get; set; }
    [Value(1, HelpText = "The ID of the instance where the sample data resides.",
        Required = true)]
    public string instanceId { get; set; }
    [Value(2, HelpText = "The ID of the database where the sample data resides.",
        Required = true)]
    public string databaseId { get; set; }
    [Value(3, Default = 0, HelpText = "The timespan in hours that will be used to filter
the "
        + "results based on a record's timestamp. The default will return the "
        + "'Top Ten' scores of all time.")]
    public int timespan { get; set; }
}
```

接著，在現有的 `InsertScoresAsync` 方法下方新增下列 `Query` 和 `QueryAsync` 方法：

```

public static object Query(string projectId,
    string instanceId, string databaseId, int timespan)
{
    var response = QueryAsync(
        projectId, instanceId, databaseId, timespan);
    response.Wait();
    return ExitCode.Success;
}

public static async Task QueryAsync(
    string projectId, string instanceId, string databaseId, int timespan)
{
    string connectionString =
        $"Data Source=projects/{projectId}/instances/"
        + $"{instanceId}/databases/{databaseId}";
    // Create connection to Cloud Spanner.
    using (var connection = new SpannerConnection(connectionString))
    {
        string sqlCommand;
        if (timespan == 0)
        {
            // No timespan specified. Query Top Ten scores of all time.
            sqlCommand =
                @"SELECT p.PlayerId, p.PlayerName, s.Score, s.Timestamp
                FROM Players p
                JOIN Scores s ON p.PlayerId = s.PlayerId
                ORDER BY s.Score DESC LIMIT 10";
        }
        else
        {
            // Query Top Ten scores filtered by the timepan specified.
            sqlCommand =
                @"SELECT p.PlayerId, p.PlayerName, s.Score, s.Timestamp
                FROM Players p
                JOIN Scores s ON p.PlayerId = s.PlayerId
                WHERE s.Timestamp >
                TIMESTAMP_SUB(CURRENT_TIMESTAMP(),
                    INTERVAL {timespan.ToString()} HOUR)
                ORDER BY s.Score DESC LIMIT 10";
        }
        var cmd = connection.CreateSelectCommand(sqlCommand);
        using (var reader = await cmd.ExecuteReaderAsync())
        {
            while (await reader.ReadAsync())
            {
                Console.WriteLine("PlayerId : "
                    + reader.GetFieldValue<string>("PlayerId")
                    + " PlayerName : "
                    + reader.GetFieldValue<string>("PlayerName")
                    + " Score : "
                    + string.Format("{0:n0}",
                        Int64.Parse(reader.GetFieldValue<string>("Score"))))
            }
        }
    }
}

```

```

        + " Timestamp : "
        + reader.GetFieldValue<string>("Timestamp").Substring(0, 10));
    }
}
}
}

```

接著，如要讓 `query` 指令正常運作，請將下列程式碼新增至程式的「Main」方法：

```

.Add((QueryOptions opts) => Query(
    opts.projectId, opts.instanceId, opts.databaseId, opts.timespan))

```

您可以使用 `dotnet-docs-samples/applications/leaderboard/step6` 目錄中的 `Program.cs` 檔案，查看啟用 `query` 指令後 `Program.cs` 檔案的範例。

現在執行程式，確認新 `query` 指令已納入程式的可能指令清單。執行下列指令：

```
dotnet run
```

您現在應該會在程式的預設輸出中看到 `query` 指令，做為新的指令選項：

```

Leaderboard 1.0.0
Copyright (C) 2018 Leaderboard

ERROR(S):
  No verb selected.

  create      Create a sample Cloud Spanner database along with sample 'Players' and
'Scores' tables in your project.

  insert      Insert sample 'players' records or 'scores' records into the database.

  query       Query players with 'Top Ten' scores within a specific timespan from
sample Cloud Spanner database table.

  help        Display more information on a specific command.

  version     Display version information.

```

現在執行 `query` 指令，查看輸入引數。輸入下列指令：

```
dotnet run query
```

這會傳回下列回應：

Leaderboard 1.0.0

Copyright (C) 2018 Leaderboard

ERROR(S):

A required value not bound to option name is missing.

--help Display this help screen.

--version Display version information.

value pos. 0 Required. The project ID of the project to use when managing Cloud Spanner resources.

value pos. 1 Required. The ID of the instance where the sample data resides.

value pos. 2 Required. The ID of the database where the sample data resides.

value pos. 3 (Default: 0) The timespan in hours that will be used to filter the results based on a record's timestamp. The default will return the 'Top Ten' scores of all time.

從回應中可以看出，除了專案 ID、執行個體 ID 和資料庫 ID 之外，還有另一個預期的引數 `value pos. 3`，可讓我們指定時間範圍 (以小時為單位)，根據 `Scores` 資料表 `Timestamp` 欄中的值篩選記錄。這個引數的預設值為 0，表示系統不會依時間戳記篩選記錄。因此，我們可以使用 `query` 指令，而不必提供「時間範圍」值，即可取得歷來「前十名」玩家的清單。

讓我們執行 `query` 指令，但不指定「時間範圍」，並使用執行 `create` 指令時所用的相同引數值。**請務必**將 `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run query PROJECT_ID cloudspanner-leaderboard leaderboard
```

畫面會顯示類似以下的回應，列出歷來排名前十的球員：

```
PlayerId : 1843159180 PlayerName : Player 87 Score : 998,955 Timestamp : 2016-03-23
PlayerId : 61891198 PlayerName : Player 19 Score : 998,720 Timestamp : 2016-03-26
PlayerId : 340906298 PlayerName : Player 48 Score : 993,302 Timestamp : 2015-08-27
PlayerId : 541473117 PlayerName : Player 22 Score : 991,368 Timestamp : 2018-04-30
PlayerId : 857460496 PlayerName : Player 68 Score : 988,010 Timestamp : 2015-05-25
PlayerId : 1826646419 PlayerName : Player 91 Score : 984,022 Timestamp : 2016-11-26
PlayerId : 1002199735 PlayerName : Player 35 Score : 982,933 Timestamp : 2015-09-26
PlayerId : 2002563755 PlayerName : Player 23 Score : 979,041 Timestamp : 2016-10-25
PlayerId : 1377548191 PlayerName : Player 2 Score : 978,632 Timestamp : 2016-05-02
PlayerId : 1358098565 PlayerName : Player 65 Score : 973,257 Timestamp : 2016-10-30
```

現在，請使用必要引數執行 `query` 指令，指定「timespan」等於一年內的時數 (8760)，查詢該年度的「前十名」玩家。**請務必**將 `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run query PROJECT_ID cloudspanner-leaderboard leaderboard 8760
```

畫面會顯示類似以下的回應，其中包含年度「前十名」玩家：

```
PlayerId : 541473117 PlayerName : Player 22 Score : 991,368 Timestamp : 2018-04-30
PlayerId : 228469898 PlayerName : Player 82 Score : 967,177 Timestamp : 2018-01-26
PlayerId : 1131343000 PlayerName : Player 26 Score : 944,725 Timestamp : 2017-05-26
PlayerId : 396780730 PlayerName : Player 41 Score : 929,455 Timestamp : 2017-09-26
PlayerId : 61891198 PlayerName : Player 19 Score : 921,251 Timestamp : 2018-05-01
PlayerId : 634269851 PlayerName : Player 54 Score : 909,379 Timestamp : 2017-07-24
PlayerId : 821111159 PlayerName : Player 55 Score : 908,402 Timestamp : 2017-05-25
PlayerId : 228469898 PlayerName : Player 82 Score : 889,040 Timestamp : 2017-12-26
PlayerId : 1408782275 PlayerName : Player 27 Score : 874,124 Timestamp : 2017-09-24
PlayerId : 1002199735 PlayerName : Player 35 Score : 864,758 Timestamp : 2018-04-24
```

注意：如果使用「timespan」執行 `query` 指令，結果可能不會列出 10 位玩家，因為我們的 Program `insert` 指令會在過去 2 年內隨機插入附有時間戳記的分數，這些分數可能位於您指定的時間範圍內，也可能不在該範圍內。您可以重新執行 `insert` 指令，插入更多分數：

```
dotnet run insert PROJECT_ID cloudspanner-leaderboard leaderboard scores
```

接著重新執行 `query` 指令，看看結果是否會發生變化。

現在執行 `query` 指令，指定「timespan」等於一個月的小時數 (730)，查詢當月「前十名」玩家。**請務必將** `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run query PROJECT_ID cloudspanner-leaderboard leaderboard 730
```

畫面會顯示當月「前十名」玩家，如下所示：

```
PlayerId : 541473117 PlayerName : Player 22 Score : 991,368 Timestamp : 2018-04-30
PlayerId : 61891198 PlayerName : Player 19 Score : 921,251 Timestamp : 2018-05-01
PlayerId : 1002199735 PlayerName : Player 35 Score : 864,758 Timestamp : 2018-04-24
PlayerId : 1228490432 PlayerName : Player 11 Score : 682,033 Timestamp : 2018-04-26
PlayerId : 648239230 PlayerName : Player 92 Score : 653,895 Timestamp : 2018-05-02
PlayerId : 70762849 PlayerName : Player 77 Score : 598,074 Timestamp : 2018-04-22
PlayerId : 1671215342 PlayerName : Player 62 Score : 506,770 Timestamp : 2018-04-28
PlayerId : 1208850523 PlayerName : Player 21 Score : 216,008 Timestamp : 2018-04-30
PlayerId : 1587692674 PlayerName : Player 63 Score : 188,157 Timestamp : 2018-04-25
PlayerId : 992391797 PlayerName : Player 37 Score : 167,175 Timestamp : 2018-04-30
```

現在，請執行 `query` 指令，指定「timespan」等於一週的小時數 (168)，查詢當週「前十名」玩家。**請務必將** `PROJECT_ID` 替換為您在本程式碼研究室一開始建立的專案 ID。

```
dotnet run query PROJECT_ID cloudspanner-leaderboard leaderboard 168
```

畫面會顯示本週「前十名」玩家，如下所示：

```
PlayerId : 541473117 PlayerName : Player 22 Score : 991,368 Timestamp : 2018-04-30
PlayerId : 61891198 PlayerName : Player 19 Score : 921,251 Timestamp : 2018-05-01
PlayerId : 228469898 PlayerName : Player 82 Score : 853,602 Timestamp : 2018-04-28
PlayerId : 1131343000 PlayerName : Player 26 Score : 695,318 Timestamp : 2018-04-30
PlayerId : 1228490432 PlayerName : Player 11 Score : 682,033 Timestamp : 2018-04-26
PlayerId : 1408782275 PlayerName : Player 27 Score : 671,827 Timestamp : 2018-04-27
PlayerId : 648239230 PlayerName : Player 92 Score : 653,895 Timestamp : 2018-05-02
PlayerId : 816861444 PlayerName : Player 83 Score : 622,277 Timestamp : 2018-04-27
PlayerId : 162043954 PlayerName : Player 75 Score : 572,634 Timestamp : 2018-05-02
PlayerId : 1671215342 PlayerName : Player 62 Score : 506,770 Timestamp : 2018-04-28
```

太棒了！

現在新增記錄時，Spanner 會將資料庫擴充至所需大小。

無論資料庫成長多少，遊戲排行榜都能透過 Spanner 及其 Truetime 技術，持續準確地擴展。

步驟 7 · 約 2 分鐘

7. 清除所用資源

盡情體驗 Spanner 的樂趣後，我們需要清理遊樂場，節省寶貴的資源和金錢。幸好這個步驟很簡單，只要前往開發人員管理中心，刪除我們在「設定 Cloud Spanner 執行個體」程式碼研究室步驟中建立的執行個體即可。

8. 恭喜！

涵蓋內容：

- 排行榜的 Google Cloud Spanner 執行個體、資料庫和資料表結構
- 如何建立 .NET Core C# 控制台應用程式
- 如何使用 C# 用戶端程式庫建立 Spanner 資料庫和資料表
- 如何使用 C# 用戶端程式庫將資料載入 Spanner 資料庫
- 如何使用 Spanner 認可時間戳記和 C# 用戶端程式庫，從資料中查詢「前十名」結果

後續步驟：

- 閱讀 [Spanner CAP 白皮書](#)
- 瞭解[結構定義設計](#)和查詢最佳做法
- 進一步瞭解 Cloud Spanner [提交時間戳記](#)

提供意見

- 請花點時間填寫這份簡短的[問卷調查](#)
-